

Backend Readiness Audit for MOS→CIR→Execute→Verify Pipeline

1. Executive Summary

The current backend is a Python (FastAPI) monolith using SQLAlchemy (Postgres) and Redis. The "Solve" pipeline is currently a mix of V1 (API-orchestrated) and V3 (Service-orchestrated) logic. The V3 solver (`SolverV3`) essentially implements a "MOS-lite" loop with a 2-pass strategy (Minimal -> Detailed) and a basic repair loop.

Readiness Status: High. The foundation for the new pipeline exists. `SolverV3` already uses a "Profile" system to select prompts/schemas, uses "Structured Outputs" validation, and has a dedicated "Repair" loop. The "CIR" (Context-Intent-Request) concept is partially present in `ProfileResolver` and `CanonicalProblem` .

Key Gap: The "Execute" phase is currently monolithic (just the LLM solving). There is no "Tool Execution" or "Sandboxed Code Execution" phase yet, other than simple plotting. The "Verify" phase exists but is LLM-based (Self-Correction/Consistency) or simple JSON schema validation, lacking a robust external verification engine.

2. Architecture Snapshot

Request Flow (Current `/solve`)

- Entry:** `POST /api/v1/solve` (`api.py:3748`)
 - Auth/Quota:** Checks `User` limits and subscription status.
 - Persistence (Pre-Solve):** None (Ephemeral until success).
- Orchestration:** `solver_v3.py` (via `get_solver_v3()`)
 - Profile Resolution:** `ProfileResolver` selects prompts/schemas based on `tier` (Free/Standard) and `mode` .
 - LLM Call:** Calls OpenAI (or fallback) with strict JSON Schema.
 - Validation:** Validates JSON against schema.
 - Repair:** If validation fails, triggers a 1-shot repair loop (`_repair_response`).
- Persistence (Post-Solve):**
 - ChatSession:** New session created.
 - SolveSession:** Immutable snapshot created for follow-up context.
 - CanonicalProblem/Solution:** De-duplicated solution stored for potential cache hits.
 - Telemetry:** `RequestEvent` and `UsageLog` recorded.

Key Components

- Orchestrator:** `backend/app/api.py` (Controller) + `solver_v3.py` (Logic).
- Database:** Postgres (SQLModel).
- Async/Queue:** Redis (used for celery workers, though `/solve` is currently `async def` and largely synchronous for the user).

3. Data Storage & Persistence (The "Ledger")

The following models are critical for the new pipeline:

Model	Purpose	Usage Status
-------	---------	--------------

SolveSession	Immutable snapshot of a solve (Problem, Steps, Answer). Used as context for follow-up.	Active. Key for "Memory".
FollowupChatTurn	Records follow-up Q&A linked to a SolveSession.	Ready. Structure exists.
LlmUsageLedger	Granular token accounting per turn/request.	Ready. Linked to turns.
CanonicalProblem	Stores normalized problems and their computed hash.	Active. Caching layer.
CanonicalSolution	Stores the "Golden" solution for a problem.	Active. Caching layer.
PromptTemplateEntry	DB-backed Prompt Registry.	Active. Versioned prompts.
PromptBinding	Links Tiers+Modes to specific Prompts+Schemas.	Active. config engine.

Artifact Storage

- **User Inputs:** Stored in `ChatMessage` (raw) and `CanonicalProblem` (normalized).
- **LLM Outputs:** Stored in `ChatMessage` (structured_data JSON) and `CanonicalSolution`.
- **Visuals:** Stored as JSON in `OCRFigure` or generated on fly.

3. Database Audit

A. Current ERD (Diagram)

```
erDiagram
    User ||--o{ ChatSession : "has"
    User ||--o{ SolveSession : "has"
    User ||--o{ UsageLog : "logs"
    User ||--o{ Upload : "uploads"
    User ||--o{ OCRJob : "requests"
    User ||--o{ VoiceSession : "initiates"
    User ||--o{ BillingLedger : "billed_in"

    ChatSession ||--o{ ChatMessage : "contains"
    ChatSession ||--o{ SolverOutputAttempt : "attempts"

    SolveSession ||--o{ FollowupChatTurn : "has_turns"
    SolveSession ||--o{ LlmUsageLedger : "tracks_usage"

    FollowupChatTurn ||--|{ LlmUsageLedger : "usage"

    Upload ||--o{ Crop : "contains"
    Crop ||--o{ OCRJob : "processed_in"
    OCRJob ||--o{ OCRArtifact : "produces"
    OCRArtifact ||--o{ OCRQuestion : "contains"
```

```

OCRArtifact ||--o{ OCRFigure : "contains"

CanonicalProblem ||--o{ CanonicalSolution : "has_solution"
CanonicalSolution ||--o{ UserSavedSolution : "saved_by"

Subscription ||--o{ Invoice : "billed_via"
Subscription ||--o{ CreditLot : "grants"

VoiceSession ||--o{ VoiceAudio : "records"
VoiceSession ||--o{ VoiceJob : "processes"
VoiceJob ||--o{ VoiceArtifact : "produces"

```

B. Table-by-Table Ledger

Table	Status	Key Purpose	Usage Notes
User	Active	Core identity & auth.	Heavily used.
ChatSession	Active	Wraps a conversation thread.	Used for /solve results (new) and history.
ChatMessage	Active	Individual message exchange.	Stores raw query & LLM response.
SolveSession	Active	Immutable snapshot of a solved problem.	Context root for follow-up chats.
FollowupChatTurn	Active	Follow-up Q&A pairs.	Linked to SolveSession.
LlmUsageLedger	Active	Fine-grained token tracking.	Logs usage for follow-ups (and soon solve).
UsageLog	Active	High-level action log.	Used for "10/minute" rate limits.
CanonicalProblem	Active	De-duplication key.	Maps Hash -> Normalized Text.
CanonicalSolution	Active	"Golden" solution cache.	Shared across users if same problem.
UserSavedSolution	Active	User bookmarks.	Explicitly saved solutions.
Upload	Active	File upload metadata.	Roots OCR pipeline.
Crop	Active	Sub-image region.	Unit of work for OCR.
OCRJob	Active	Async processing task.	Tracks status of OCR extraction.
OCRArtifact	Active	Result of OCR.	Contains Markdown, LaTeX, JSON.
OCRQuestion	Active	Parsed question entity.	Extracted from Artifact.
OCRConfirmation	Active	User-verified text.	"Human in loop" confirmation.
SystemConfig	Active	Global dynamic settings.	Pricing, feature flags.
PromptTemplateEntry	Active	Versioned prompts.	DB-backed Prompt Registry.
PromptBinding	Active	config engine.	Links Tiers to Prompts.

Subscription	Active	User plan status.	Core billing entity.
BillingLedger	Active	Virtual currency log.	Credits, Debits, Fees.
Invoice	Active	Stripe invoice mirror.	Generated for subscriptions/topups.
School	Active	School directory.	Location context for Curriculum.
VoiceSession	Active	Voice mode root.	Container for audio interaction.

C. Orphan / Unused Tables

Table	Status	Reason	Recommendation
SchoolImportRun	Audit	Only used during import scripts.	Keep for audit, but low runtime value.
DeviceSignupLog	Legacy/Audit	Likely for abuse prevention.	Verify if actively written to.
ReconciliationFinding	Future	Framework for billing integrity.	Current usage likely low/zero.
InvoiceSequence	Utility	Atomic counter.	Tech utility, not business domain.

4. Prompting & Schema System

- **Registry:** Fully DB-backed (`prompt_registry_service`).
- **Resolver:** `ProfileResolver` maps (User Tier, Request Mode) -> `PromptBinding` -> (System Prompt, Developer Prompt, Output Schema) .
- **Schema Enforcement:** Uses `Draft202012Validator` for pre-validation and `OpenAI json_schema` (Strict Structured Outputs) for generation.
- **Token Accounting:** `TokenPolicy` (DB-backed) defines limits. `LlmUsageLedger` tracks actual usage.

5. Gap Analysis (vs. Target MOS→CIR→Execute→Verify)

Feature	Current State	Target State	Gap / Action
MOS (Model Orchestration)	<code>SolverV3</code> handles Profile selection and basic fallback.	Advanced Routing (Model selection based on difficulty/domain).	Refactor: Enhance <code>ProfileResolver</code> to use "Router" logic based on classification.
CIR (Context-Intent-Request)	<code>CanonicalProblem</code> has basic "Intent".	Deep understanding of intent to select specialized "Executors".	New Component: Explicit "Intent Classifier" step before Solver.
Execute (Tools)	Limited to internal "Plotting".	WolframAlpha, Python Sandbox, Search.	Major Gap: Need a "Tool Use" or "Code Interpreter" loop.
Verify (Robustness)	Self-Consistency (LLM-based) & Schema	Code-based verification, Unit Test generation.	Major Gap: Need a "Verifier" agent/module that

	Validation.		runs code.
Pipeline Structure	Linear: Check -> Solve -> Save.	Cyclic: Plan -> Execute -> Verify -> Refine.	Refactor: Convert <code>solve()</code> into a loop/graph (e.g., LangGraph style or custom loop).

6. Recommendations & Plan

Recommended Starting Point

Do **NOT** rewrite `SolverV3` from scratch. It is a solid foundation. Instead, **Refactor `SolverV3` into a `PipelineExecutor`**.

Phased Implementation Plan (2 Weeks)

Phase 1: MOS & CIR Refinement (Days 1-5)

- 1. Extract Router:** Pull routing logic out of `ProfileResolver` into a dedicated `ModelOrchestrationService` (MOS).
- 2. Intent Classification:** Add a lightweight "Classifier" step that determines if the problem is "Math", "Writing", "Search", or "Analysis".
- 3. Refactor Solve Pipeline:** Break `solve()` in `solver_v3.py` into distinct steps: `classify()`, `select_profile()`, `generate_plan()`.

Phase 2: Execution & Verification (Days 6-10)

- 1. Tool Interface:** Define a standard `Tool` interface.
- 2. Python Sandbox:** Implement a safe code execution environment (or mock it for V1).
- 3. Verify Loop:** Add a "Verification" step after generation. If `verify(result) == fail`, trigger `repair()` loop with verification feedback (not just schema error).

Phase 3: Connect & Deploy (Days 11-14)

- 1. Update `/solve` endpoint:** Point to the new Pipeline.
- 2. Migration:** Ensure `SolveSession` and `Canonical` models are populated correctly by the new pipeline.
- 3. Testing:** Run regression tests on "Hard" problems to verify improvement.

7. Evidence Appendix (Required)

A. Relevant Code Modules

- API Entry Point:** [backend/app/api.py](#) (Main `/solve` handler)
- Solver Logic:** [backend/app/services/solver_v3.py](#) (Orchestration & Validation)
- Data Models:** [backend/app/models.py](#) (SQLModel definitions)
- Prompt Selection:** [backend/app/llm_profiles/profile_resolver.py](#) (Binding logic)
- Registry Service:** [backend/app/services/prompt_registry_service.py](#) (DB Access)

B. Representative Request/Response Logs (Sanitized)

Request (`POST /api/v1/solve`)

```
{
  "user_id": 123,
```

```
"text_query": "Solve for x: 2x + 5 = 15",
"image_url": null,
"subject": "Algebra",
"mode": "detailed",
"features_used": {
  "calculator": false,
  "canvas": true
},
"trusted_context": {
  "learning_mode": "solve",
  "grade_level": "10"
}
}
```

Response (200 OK)

```
{
  "session_id": 456,
  "solve_session_id": 789,
  "solution": {
    "final_answer": "x = 5",
    "steps": [
      {
        "explanation": "Subtract 5 from both sides.",
        "math": "2x = 10"
      },
      {
        "explanation": "Divide by 2.",
        "math": "x = 5"
      }
    ]
  },
  "model_used": "gpt-4o",
  "tokens_used": 450,
  "telemetry": {
    "provider": "openai",
    "latency_ms_total": 2300,
    "validated": true
  }
}
```

C. Representative Stored Database Rows

SolveSession (Context Root)

```
{
  "id": 789,
  "user_id": 123,
  "problem_text": "Solve for x: 2x + 5 = 15",
  "topic": "Algebra",
}
```

```
"solution_steps_text": "Step 1: Subtract 5...\nStep 2: Divide by 2...",
"final_answer_text": "x = 5",
"created_at": "2024-02-07T14:30:00Z"
}
```

PromptBinding (Configuration)

```
{
  "id": "bind-uuid-1",
  "tier": "STANDARD",
  "mode": "SOLVE",
  "global_system_prompt_id": "sys-prompt-v2",
  "developer_prompt_id": "dev-prompt-algebra",
  "output_schema_id": "schema-v1-strict",
  "max_output_tokens": 4096,
  "trim_strategy": "trim_context_first"
}
```

CanonicalProblem (De-duplication)

```
{
  "id": 101,
  "normalized_problem_hash": "sha256_hash_of_normalized_text",
  "normalized_text": "solve for x 2x + 5 = 15",
  "intent": "math_solve",
  "seen_count": 42
}
```

D. Schema Files

- **Status:** No static JSON/YAML files found.
- **Mechanism:** Schemas are stored in the database table `JsonSchemaEntry` and loaded dynamically by `ProfileResolver`.
- **Key File:** [backend/app/models.py](#) (Lines 721-733 define `JsonSchemaEntry`).